CSE247 Data Structures (Fall'25)

Final Examination - Solutions

Max Marks: 80 *Time Allowed:* 3 hours

Instructions:

- Answer the questions in the spaces provided on the question sheets.
- Extra sheets may be used for rough work, but make sure to write your name and ERP on them. Answers on extra sheets will not be graded.
- Understanding the questions is part of the exam, please do not ask for clarifications during the exam.
- While describing algorithms, you may use pseudocode or a mix of pseudocode and C++ code.

Name: _____ **ERP:** _____

Question:	Hashing	BFS / DFS	MSTs / SSCs	Shortest paths	Misc.	Total
Marks:	9½	26	8	14½	22	80
Score:						

Question 1: Hashing ..... 9½ marks

(a) [3 marks] The following C++ code implements the put operation for a hash table using Linear Probing (an open addressing scheme). The table size is `m`, `keys` is vector storing the keys. `vals` is a vector storing the values, `filled` is a boolean vector where `filled[i]` is `true` if slot `i` is occupied. `hash(key)` is a function that returns the initial hash index (an integer between 0 and $m-1$).

Complete the `put` function by selecting the correct option (letter) from the options table below right for each placeholder. You may use each letter once, more than once, or not at all. No other code is allowed.

```
1  void put(const Key& key, const Value& val) {
2      int i;
3      for (i = [ E ]; [ C ]; i = [ A ])
4          if ( [ B ] )
5              break;
6      keys[i] = [ D ];
7      vals[i] = [ F ];
8      filled[i] = true;
9  }
```

Options table

A	`(i+1) % m`
B	`keys[i] == key`
C	`filled[i]`
D	`key`
E	`hash(key)`
F	`val`

(b) A hash table of size $m = 13$ uses the hash function $h(k) = k \mod 13$ and resolves collisions using linear probing. The table currently contains the 8 keys shown below.

keys[]	37	-	-	-	43	17	-	20	33	46	-	11	24
	0	1	2	3	4	5	6	7	8	9	10	11	12

i. [4 marks] Which of the following sequences of insertions of these 8 keys could have resulted in this exact arrangement of keys? (Select all that apply)

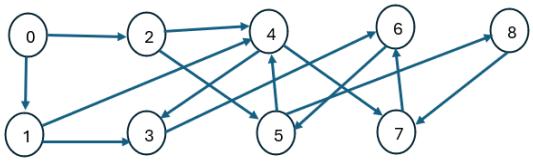
- ◯ 17, 43, 20, 33, 46, 11, 24, 37
- ✓ 43, 17, 20, 33, 46, 11, 24, 37
- ✓ 43, 20, 11, 17, 33, 24, 46, 37
- ◯ 43, 17, 20, 33, 46, 11, 37, 24

ii. [2½ marks] Under the *uniform hashing assumption*, where is the next key least likely to be added in this linear-probing hash table (no resizing)? Mark all such indices (from the empty slots).

◯	✓	✓	◯	◯
1	2	3	6	10

Question 2: BFS / DFS ..... 26 marks

(a) Consider the directed graph G (drawn below) with $V = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$ and $E = \{0 \to 1, 0 \to 2, 1 \to 3, 1 \to 4, 2 \to 4, 2 \to 5, 3 \to 6, 4 \to 3, 4 \to 7, 5 \to 4, 5 \to 8, 6 \to 5, 7 \to 6, 8 \to 7\}$. Assume the adjacency lists are in sorted order: for example, when iterating over the edges leaving vertex 4, consider the edge $4 \to 3$ before $4 \to 7$.



Depth-First Search (DFS) is run starting from vertex 0.

i. [2 marks] List the vertices in the exact order they are first discovered by DFS.

0	1	3	6	5	4	7	8	2

ii. [2 marks] What is the maximum depth of the recursion stack during DFS execution? 7

iii. [2 marks] Write the path returned by DFS-based path finding from vertex 0 to vertex 8.

Solution: $0 \to 1 \to 3 \to 6 \to 5 \to 8$

iv. [2 marks] Is the path from (iii) the shortest path from 0 to 8? If not, give a shorter valid path.

Solution: No, the shortest path is $0 \to 2 \to 5 \to 8$.

(b) Breadth-First Search (BFS) is run starting from vertex 0 on the same diagraph from part (a).

i. [2 marks] List the vertices in the exact order they are dequeued from the BFS queue.

0	1	2	3	4	5	6	7	8

ii. [2 marks] BFS and DFS both explore all vertices reachable from the source. Explain why BFS is guaranteed to compute a shortest path from vertex 0 to every reachable vertex in this graph, whereas DFS is not.

Solution: BFS explores vertices level by level, guaranteeing shortest paths in unweighted graphs. DFS explores deeply first and does not account for path length, so it may return longer paths even when shorter ones exist.

(c) [3 marks] Assume a DAG where DFS is run starting from vertex 0, but the graph is disconnected, resulting in multiple DFS trees. Does the concatenation of reverse post-orders from multiple DFS trees still produce a valid topological ordering? Justify your answer.

Solution: Yes, concatenating reverse post-orders from multiple DFS trees in a disconnected DAG still produces a valid topological ordering. Each DFS tree explores a separate component of the graph, and since there are no cycles in a DAG, the relative order of vertices within each tree is preserved. When concatenated, the overall order maintains the property that for every directed edge $A \to B$, vertex A appears before vertex B in the final ordering.

(d) [3 marks] A software company is designing a task scheduling system for a large project. Each task is represented as a vertex in a directed graph, and a directed edge $A \to B$ means task A must be completed before task B. The scheduling system uses the DFS-based topological sorting algorithm, which outputs tasks in reverse DFS post-order. During testing, the system is run on the following dependency graph:

Tasks: $\{A, B, C\}$

Dependencies: $\{A \to B, B \to C, C \to A\}$

The system successfully completes DFS and outputs the order C, B, A. Explain why the ordering produced by the system is not a valid task schedule, even though the algorithm completed without errors.

Solution: The ordering C, B, A is not valid because the dependency graph contains a cycle: $A \to B \to C \to A$. In a cyclic graph, there is no way to satisfy all dependencies, as each task depends on the completion of another task in the cycle. Therefore, no valid topological ordering exists for this graph, and the output from the DFS-based algorithm does not represent a feasible task schedule.

(e) A student implements topological sort as follows:

```
vector<int> topoSort(Digraph& G) {
    vector<bool> marked(G.V(), false);
    vector<int> result;
    for (int v = 0; v < G.V(); v++) {
        if (!marked[v])
            dfs(G, v, marked, result);
    }
    return result;
}
```

```
void dfs(Digraph& G, int v, vector<bool>& marked,
    vector<int>& result) {
    marked[v] = true;
    result.push_back(v);
    for (int w : G.adj(v)) {
        if (!marked[w])
            dfs(G, w, marked, result);
    }
}
```

i. [4 marks] For the DAG: $0 \to 1$, $0 \to 2$, $1 \to 3$, $2 \to 3$, trace through this code starting from vertex 0. Show what the result contains after execution.

Solution: The DFS explores as follows:

1. Start at vertex 0: mark 0 and add to result: result = [0]
2. Explore neighbor 1: mark 1 and add to result: result = [0, 1]
3. Explore neighbor 3: mark 3 and add to result: result = [0, 1, 3]
4. Backtrack to 1, then backtrack to 0.
5. Explore neighbor 2: mark 2 and add to result: result = [0, 1, 3, 2]

The final result after execution is: [0, 1, 3, 2]

ii. [4 marks] Verify whether the output is correct for the given DAG. If incorrect, identify the issue and provide the corrected code.

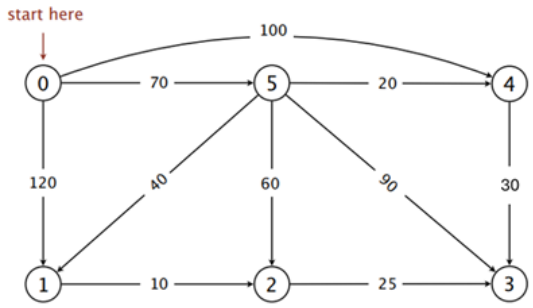
Solution: The output [0, 1, 3, 2] is not a valid topological order because vertex 2 appears after vertex 3, violating the dependency $2 \to 3$. The issue is that the algorithm adds vertices to the result in pre-order rather than reverse post-order. To fix this, we should add vertices to the result after exploring all their neighbors to ensure they are added in post-order in the DFS traversal. Second, we need to reverse the result before returning it in the `topoSort` function. The corrected code is:

```
vector<int> topoSort(Digraph& G) {
    vector<bool> marked(G.V(), false);
    vector<int> result;
    for (int v = 0; v < G.V(); v++) {
        if (!marked[v])
            dfs(G, v, marked, result);
    }
    reverse(result.begin(), result.end());
    return result;
}
```

```
void dfs(Digraph& G, int v, vector<bool>&
    marked, vector<int>& result) {
    marked[v] = true;
    for (int w : G.adj(v)) {
        if (!marked[w])
            dfs(G, w, marked, result);
    }
    result.push_back(v);
}
```

Question 3: MSTs / SSCs ..... 8 marks

(a) Consider the following edge-weighted digraph G:



List the weights of the MST edges in the order that Prim's algorithm (lazy implementation) adds them to the MST. Start Prim's algorithm from vertex 1.

____ ____ ____ ____ ____

(b) [3 marks] Kruskal's minimum spanning tree algorithm sort all edges first to quickly find the lowest cost edge not already chosen in the main loop. What would be the running time of the algorithm if, instead of sorted array, it used a simple, unsorted array to store the edges?

Solution: If Kruskal's algorithm used an unsorted array to store the edges, it would need to scan through all edges each time to find the minimum weight edge not already included in the MST. This would result in a running time of $O(E^2)$, where E is the number of edges.

(c) [5 marks] Given the following directed graph G:

Vertices: 0, 1, 2, 3, 4, 5, 6

Edges: $0 \to 1,\ 1 \to 2,\ 2 \to 0,\ 2 \to 3,\ 3 \to 4,\ 4 \to 5,\ 5 \to 3,\ 5 \to 6,\ 6 \to 6$

Execute the *Kosaraju-Sharir algorithm* from the lectures on this graph to identify the strongly connected components in the order they are discovered. Also, show the final `id[]` array.

Solution: Phase 1: Reverse post-order of the reversed graph:

0, 1, 2, 3, 4, 5, 6

Phase 2: DFS on original graph in the above order:

- Start DFS from vertex 0: discovers SCC $\{0, 1, 2\}$
- Next, start DFS from vertex 3: discovers SCC $\{3, 4, 5\}$
- Finally, start DFS from vertex 6: discovers SCC $\{6\}$

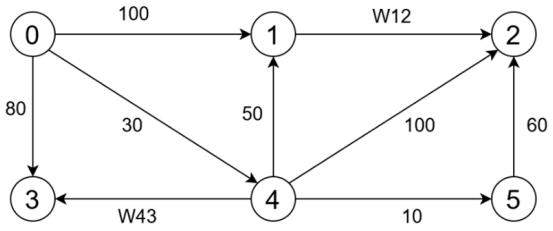
The final `id[]` array is:

`id[]`	0	1	2	3	4	5	6
	0	0	0	1	1	1	2

Question 4: Shortest paths ..... 14½ marks

(a) For the following digraph (G), find positive integer values for W_{12} and W_{43} such that when running Dijkstra's shortest path algorithm starting from vertex 0, the following two conditions are met:

- *Removal Order:* The vertices are removed from the priority queue in the following exact order: 0, 4, 5, 3, 1, 2
- *Distance Updates:* The shortest path distance (`distTo`) for vertices 1, 3, and 2 must each be updated exactly twice (not counting the initial ∞).



i. [3 marks] What is the range for W_{43}? $\boxed{10} < W_{43} < \boxed{50}$

ii. [2 marks] What is the minimum possible positive integer value for W_{12}? $\boxed{20}$

iii. [2½ marks] For which vertices can the `distTo` values be updated three times by Dijkstra's shortest path algorithm (not counting the initial ∞)?

0	1	2	3	4	5
◯	◯	✓	◯	◯	◯

(b) Consider running the Bellman-Ford algorithm on the digraph G shown above, with $W12 = W_{43} = 5$ and source vertex $s = 0$. Assume that, within a pass, the edges are relaxed in sorted order: $0 \to 1$, $0 \to 3$, $0 \to 4$, $1 \to 2$, $4 \to 1$, $4 \to 2$, $4 \to 3$, $4 \to 5$, $5 \to 2$

i. [2½ marks] After completing the first pass, what are the values of `distTo[v]` for each vertex `v`? Write the values in the corresponding boxes.

`distTo[]`	0	80	100	35	30	40
	0	1	2	3	4	5

ii. [2½ marks] After completing the first pass, for which vertices `v` is `distTo[v]` the length of the shortest path from `s` to `v` (i.e., `distTo[v]` will not change in the subsequent passes)? Mark all vertices that apply.

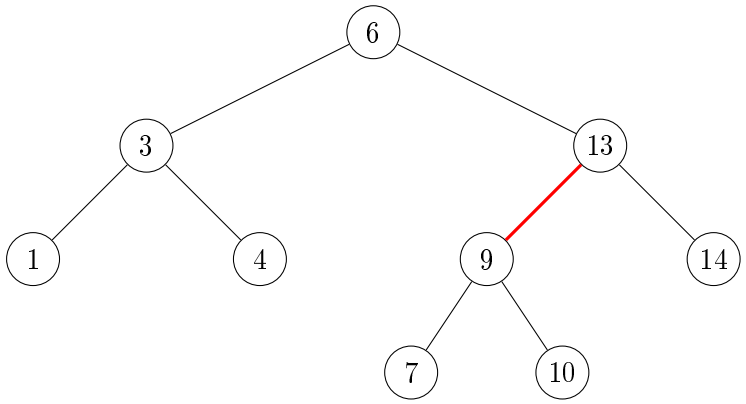
0	1	2	3	4	5
✓	✓	◯	✓	✓	✓

iii. [2 marks] Which two edges (from the ordered list given above) can be swapped such that `distTo[v]` becomes the length of the shortest path from `s` to `v` for all vertices `v` immediately after the first pass (i.e., `distTo[v]` will not change in the subsequent pass)? $\boxed{1 \to 2 \text{ and } 4 \to 1}$

Question 5: Misc. ..... 22 marks

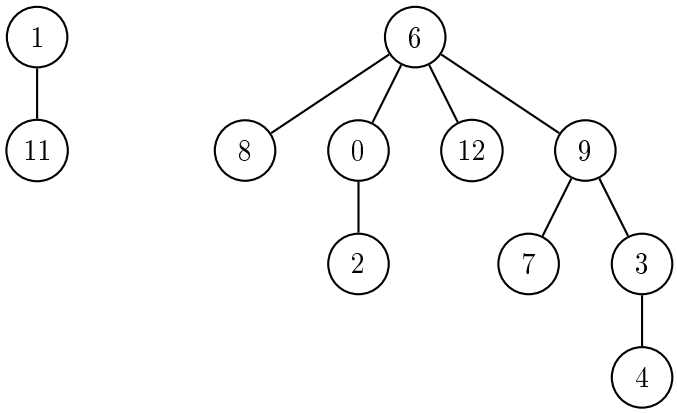
(a) [5 marks] Draw the left-leaning red-black tree if keys are inserted in the following order: $9, 3, 10, 1, 6, 14, 4, 7, 13$.

Solution:



(b) [5 marks] Consider the set of initially unrelated elements $0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12$. Draw the final forest of trees that results from the following sequence of operations using *union-by-size*. Break ties by keeping the first argument as the root. `Union(0,2)`, `Union(3,4)`, `Union(9,7)`, `Union(9,3)`, `Union(6,8)`, `Union(6,0)`, `Union(12,6)`, `Union(1,11)`, `Union(9,6)`.

Solution:



(c) [6 marks] Assume that a *minimum binary heap* stores n items. Describe and estimate the costs of procedures to

i. find the parent and offsprings of a given node.

> **Solution:** Parent of node i is $\lfloor (i-1)/2 \rfloor$. Left child is $2i+1$, right child is $2i+2$.

ii. delete the topmost item from a non-empty heap.

> **Solution:** Remove the root, replace it with the last element in the heap, and then restore the heap property by "heapifying down". As the height of the heap is $O(\log n)$, the time complexity is $O(\log n)$.

iii. Starting from an array holding n items in arbitrary order, rearrange those items so that they form a heap.

> **Solution:** Use a bottom-up approach: starting from the last non-leaf node, apply "heapify" to each node in reverse order. This process takes $O(n)$ time. Or the top-down approach: insert each item into the heap one by one, which takes $O(n \log n)$ time.

(d) [2 marks] A stable sorting method is one where items whose keys compare as equal will appear in the output in the same order that they appeared in the input list. Would a heap sort based on the algorithms you have documented be a stable sorting algorithm? Justify your answer.

> **Solution:** No, heap sort is not a stable sorting algorithm. In heap sort, elements are moved around during the heapify process, which can change the relative order of equal elements.

(e) [4 marks] A certain program has to maintain an array, count, of n counters which are all initialised to zero. The value of counter i can be incremented by one by the call: `increment(i)`, and this is the only way the program changes counter values. Two variables, `mincount` and `maxcount`, must always hold the smallest and largest of the counter values whenever the point of execution is not within the function increment. You may assume that increment is called about $1000n$ times when the program is run and that its argument is typically uniformly randomly distributed between 1 and n, but on some runs it cycles through the numbers 1 to n in order 1000 times. Describe, in detail, an efficient data structure and algorithm to use when

i. n is expected to be about 10.

> **Solution:** Use a simple array of size n to store the counters. Keep track of min and max values by scanning the array when needed.

ii. n is about a million.

> **Solution:** Use a balanced binary search tree (e.g., LLRB tree) to store the counters. Each node stores a counter value and its frequency. Keep track of min and max values in the tree.

iii. Suppose your algorithm for (i) above were used when $n = 10^6$, estimate how much slower it would be compared with your algorithm for (ii).

> **Solution:** The algorithm for (i) has a time complexity of $O(n)$ for each increment operation, leading to $O(1000n^2)$ for $1000n$ increments. For $n = 10^6$, this results in $O(10^{12})$ operations. The algorithm for (ii) has a time complexity of $O(\log n)$ for each increment, leading to $O(1000n \log n)$ operations. For $n = 10^6$, this results in approximately $O(2 \times 10^7)$ operations. Thus, the algorithm for (i) would be significantly slower, by a factor of about 5×10^4.